# Comparative Analysis of Reinforcement Learning Architectures and Reward Structures in the Snake Game

**Francisco Alejandro Bernal Cañez (ID: 2151939)**


## Abstract

This work studies Deep Reinforcement Learning (DRL) for the Snake game modeled as a Markov Decision Process (MDP). We evaluate Deep Q-Networks (DQN), Proximal Policy Optimization (PPO), and Advantage Actor-Critic (A2C) using both Multi-Layer Perceptrons (MLP) and Convolutional Neural Networks (CNN). Experiments analyze performance under full and partial observability. Results show that DQN achieves the best terminal precision in fully observable settings, while PPO demonstrates superior robustness under partial observability. All learned agents outperform heuristic baselines.


## 1. Introduction

Reinforcement Learning (RL) addresses sequential decision-making problems where agents learn optimal behavior through interaction with the environment. The game *Snake* provides a suitable benchmark, requiring a balance between short-term survival and long-term planning.

This project evaluates deep RL methods: Deep Q-Network (DQN), Advantage Actor-Critic (A2C), and Proximal Policy Optimization (PPO), on a $7 \times 7$ grid. We compare observability settings (full vs. partial) and neural architectures (MLP vs. CNN) to study the trade-off between performance and robustness. This report details the training configuration and evaluates agent performance across both obstacle-free environments and scenarios involving stochastic obstacles.

## 2. Problem Formulation

The Snake environment is modeled as a Markov Decision Process $(S, A, P, R)$:

- **State Space ($S$):** States encode the positions of the snake (head and body), food, and boundaries, either as a grid or a flattened vector. Under partial observability, the agent receives a $3 \times 3$ or $5 \times 5$ visibility window centered on the head (See Figure 4).
- **Action Space ($A$):** Four discrete actions: *Up, Down, Left, Right*.
- **Transition Dynamics ($P$):** Deterministic movement by one tile. Eating food increases length and respawns fruit randomly; collisions with walls or the body terminate the episode.
- **Reward Function ($R$):** Positive rewards for food consumption and strong penalties for collisions.

*Figure 1.* Fully Observable

*Figure 2.* Partially Observable Mask = 1

*Figure 3.* Partially Observable Mask = 2

*Figure 4.* Snake board with different observability settings. The yellow square represents the snake's head and the green square represents the fruit.

## 3. Methodology

### 3.1. Environment Design and Custom Wrappers

While the core game logic was managed by a custom engine via `SnakeFullEnv` and `SnakePartialEnv`, we implemented four Gymnasium-compliant environments to serve as specialized wrappers. This architectural layer was essential for interfacing the game state with the **Stable-Baselines3** library and for standardizing observations across different model-input requirements. The MLP wrappers (`Snake_Fully_Observable` and `Snake_Partially_Observable`) provide flattened $1 \times N$ vectors, whereas the CNN wrappers (`CNN_Snake_Fully_Observable` and `CNN_Snake_Partially_Observable`) implement a transpose to a $(C, H, W)$ channels-first format ($4 \times 7 \times 7$). This spatial formatting enables the 2D convolutional layers to effectively extract proximity correlations and geometric patterns from the grid.

**Observation Tensors** States are encoded as 4-channel tensors providing a semantic grid view: **Ch.0:** Walls/Obstacles; **Ch.1:** Fruit; **Ch.2:** Body; **Ch.3:** Head.

In Partially Observable settings the visibility is centered in the snake's head. To maintain a global objective gradient, we append a normalized relative fruit vector $[\Delta x, \Delta y]$ to the MLP inputs, ensuring the agent retains directional guidance even when the fruit is outside the local mask.

### 3.2. Reward Shaping

The reward function mitigates sparsity by providing dense feedback (Table 1).

where the specific values for each component are detailed in Table 1.

| Event | Value | Rationale |
|---|---|---|
| Fruit | $1.2$ | Objective |
| Win | $3.0$ | Completion |
| Collision | $-1.2$ | Failure |
| Step | $-0.03$ | Efficiency |
| Timeout | $-0.5$ | Avoid stagnation |

*Table 1.* Reward configuration.

**Incentive Design** Equal fruit and collision magnitudes balance risk, while the step penalty discourages long trajectories. A timeout at $2L^2$ steps prevents survival without progress.

## 4. Experimental Setup

### 4.1. Baseline Policies

We evaluate two reference policies:

- **Random:** uniformly samples actions at each timestep.
- **Heuristic:** Serves as a non-learning benchmark designed to simulate greedy human-like navigation. Rather than simple random movement, it employs a two-stage decision hierarchy: first, it identifies a candidate set of actions that minimize the Manhattan distance to the target fruit; second, it applies a safety filter to prune any actions resulting in immediate terminal collisions (walls or body segments). If the intersection of safe and distance-reducing actions is non-empty, the agent samples from these optimal moves. If no progress can be made safely, it defaults to a fallback survival move from the available safe actions. Finally, if there are no safe movements available, it just dies with dignity going *Up*. This ensures the baseline provides a challenging performance floor that requires RL agents to learn both spatial efficiency and collision avoidance.

### 4.2. Agent Architectures

We evaluated three algorithms representing distinct paradigms in Reinforcement Learning. This selection allows us to compare the trade-offs between sample-efficient off-policy learning and stable on-policy gradient updates within a constrained grid environment:

- **DQN (Deep Q-Network):** A value-based, off-policy method that approximates the optimal action-value function $Q^*(s, a)$. It utilizes an Experience Replay buffer to break temporal correlations and a Target Network to stabilize training. It serves as our representative for deterministic, value-centric learning.
- **A2C (Advantage Actor-Critic):** A synchronous, on-policy framework that updates both a policy (actor) and a value function (critic). It leverages the advantage signal $A(s, a)$ to reduce variance in gradient updates, providing a baseline for standard actor-critic performance.
- **PPO (Proximal Policy Optimization):** A state-of-the-art on-policy algorithm that employs a clipped objective function to constrain policy updates. This prevents catastrophic performance collapses, making it the primary candidate for evaluating stability and robustness under partial observability.

**Feature Extraction Architectures** We evaluate two distinct neural paradigms for processing the state space:

- **Multi-Layer Perceptron (MLP):** Serves as our baseline architecture, utilizing fully connected layers with varying depths (e.g., $[256, 256]$ or $[256, 128, 64]$ depending on observability). The MLP treats the flattened grid as a high-dimensional vector, learning direct mappings between entity positions and optimal actions.
- **Convolutional Neural Network (CNN):** Designed to exploit the spatial correlations inherent in the $7 \times 7$ grid. The architecture consists of two convolutional layers using $3 \times 3$ kernels, a stride of $1$, and padding of $1$. This configuration is specifically chosen to preserve spatial dimensions (avoiding information loss at the edges) before passing the features through a flattening layer to a $128$-unit linear bottleneck.

### 4.3. Hyperparameter Configuration

The hyperparameter selection was not automated via search algorithms but was instead empirically tuned through iterative observation of agent behavior across different environmental constraints. This manual refinement process allowed

for specific adjustments based on the qualitative failures observed during early training phases.

**Empirical Adjustments for Partial Observability** Transitioning from full to partial observability (Mask = 1 and Mask = 2) introduced significant state uncertainty, necessitating a heuristic-driven recalibration of the learning dynamics:

- **Exploration-Exploitation Balance:** We observed that agents under restricted vision frequently converged to sub-optimal "survival loops." To counteract this, the Entropy Coefficient for PPO and A2C was empirically increased (reaching $0.25$ for A2C) to penalize premature policy certainty. In DQN, the exploration fraction was extended to $0.50$ for Mask 1, ensuring the agent continued to sample the environment long enough to discover the fruit-reward signal despite its limited field of view.
- **Stability Tuning:** To account for the high variance in local observations, the DQN Replay Buffer was doubled to $2 \times 10^5$. We observed that a lower learning rate ($5 \times 10^{-4}$) was required to prevent the policy from oscillating when the snake's tail entered and exited the egocentric mask.
- **Architectural Bottlenecking:** For CNN-based policies, we empirically reduced the MLP decision head to $[64, 64]$. This forced the network to prioritize the spatial features extracted by the convolutional layers, as larger heads were observed to overfit to specific coordinate-action pairs rather than general spatial patterns.

*Table 2.* Empirically Tuned DQN Configurations

| Hyperparameter | Full | Mask 1 | Mask 2 |
|---|---|---|---|
| Learning Rate | $10^{-3}$ | $5 \times 10^{-4}$ | $5 \times 10^{-4}$ |
| Exploration Frac. | 0.15 | 0.50 | 0.15 |
| Buffer Size | $10^5$ | $2 \times 10^5$ | $2 \times 10^5$ |
| Net Arch (MLP) | [256, 256] | [256, 128, 64] | |

*Table 3.* Baseline Policy-Gradient Configurations

| Hyperparameter | PPO | A2C |
|---|---|---|
| Learning Rate | $3 \times 10^{-4}$ | $7 \times 10^{-4}$ |
| n_steps | 1024 | 32 |
| Ent. Coeff. | 0.01 | 0.01 |

## 5. Results and Discussion

We evaluate the performance-robustness trade-offs of our agents by comparing algorithms, architectures, and observability settings. To ensure statistical significance and reliability, each model was evaluated by simulating 100 independent episodes. From these simulations, we extracted key metrics including mean cumulative reward, average fruit consumption (success rate), and episode termination causes (wall collisions, self-collisions, or timeouts). This comprehensive evaluation framework allows us to assess not only the peak performance of the agents but also their consistency and failure modes across diverse environmental configurations..

### 5.1. MLP Architectures under Varying Observability

We evaluate DQN, PPO, and A2C trained with MLPs across full and partial observability.

As illustrated in Figure 5, PPO exhibits superior sample efficiency, surpassing the heuristic baseline significantly faster than its counterparts. While PPO achieved the highest rewards, it fluctuated late in training. DQN and A2C showed greater instability. In particular, A2C suffered total performance collapses, regressing to negative mean rewards, before eventually recovering.

A critical observation, which remained a consistent trend across every experiment in this study, is the temporal disparity shown in Figure 6. While PPO and A2C converge rapidly in wall-clock time, DQN takes significantly longer to train. This repeated behavior is attributed to the computational overhead of managing a large replay buffer and the frequent synchronization required for target network updates, making DQN the most time-intensive algorithm to deploy despite its off-policy nature.

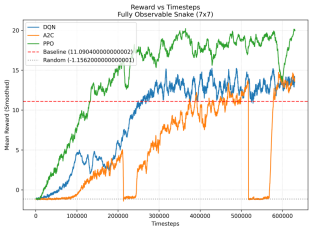


*Figure 5.* Steps

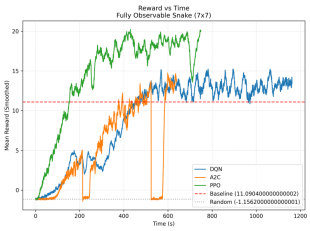


*Figure 6.* Time (s)

*Figure 7.* Training performance in the fully observable setting.

Despite PPO's faster learning, evaluation results (Table 4) show that only DQN achieves successful task completion (15% win rate). PPO and A2C reach near-optimal fruit counts (up to 23) but fail at terminal states.

This gap reflects a precision issue: near full occupancy, optimal behavior requires exact trajectory planning. PPO and A2C, being stochastic, are prone to final-step errors, whereas DQN's deterministic policy enables consistent completion.

**Partial Observability** Under limited vision (Fig. 10), PPO remains robust, while DQN degrades significantly. PPO even surpasses the fully observable heuristic baseline, indicating effective local decision-making despite missing global information.

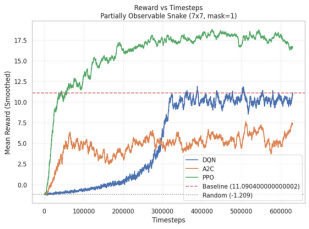


*Figure 8.* Mask = 1

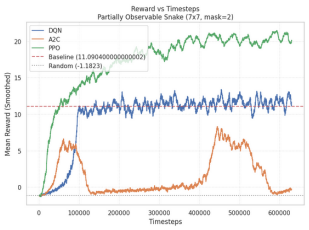


*Figure 9.* Mask = 2

*Figure 10.* Training under partial observability.

This superiority was not limited to the training phase but was definitively confirmed during the evaluation process. While DQN became increasingly unstable under uncertainty, PPO maintained higher fruit counts and lower collision rates during testing, proving to be the clear winner in partially observable settings. As illustrated by the distribution in Fig. 11, PPO (Mask = 2) achieves a consistency and median performance that rivals the fully observable DQN. Ultimately, while DQN excels as a specialist with full information, PPO provides a more robust and reliable solution under uncertainty, generalizing better in both convergence speed and real-world deployment performance.

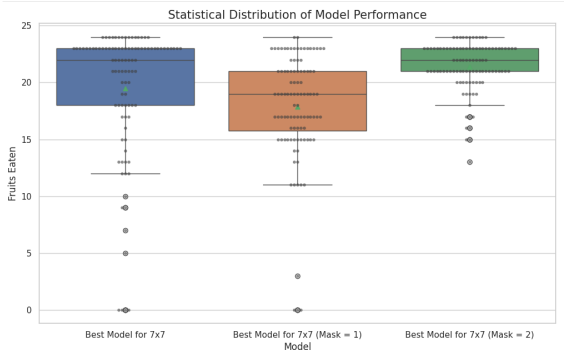


*Figure 11.* Performance distribution across observability settings.

### 5.2. Reward Shaping Analysis

We analyze the impact of reward design using DQN under five configurations (Table 5), keeping all hyperparameters fixed.

DQN remains robust across all meaningful configurations (Default, NSP, Original, Aggressive), consistently reaching full board completion. However, behavior varies significantly with reward structure:

- **Default vs. No Step Penalization (NSP):** Removing the negative step penalty causes a sharp increase in timeouts (8% $\rightarrow$ 60%). This confirms that constant temporal pressure is essential to discourage inefficient looping and force the agent toward the fruit.
- **Reward scaling (Original vs. Aggressive):** Both achieve high performance, but aggressive rewards slightly increase self-collisions, indicating a trade-off between speed and safety.
- **Failure case (Survival):** The agent converges to looping behavior with 100% timeouts, prioritizing survival over fruit collection due to misaligned incentives.

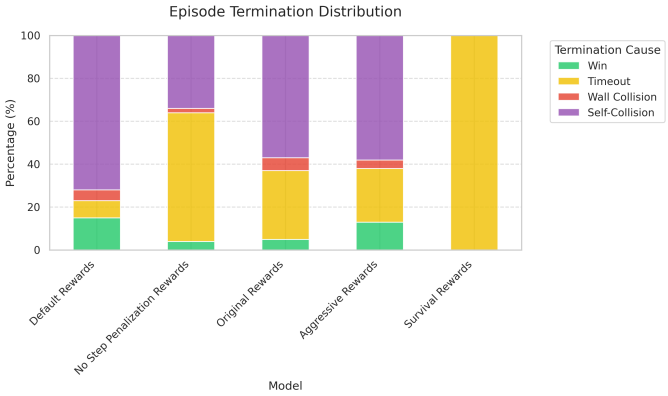


*Figure 12.* Episode termination distribution across reward configurations.

### 5.3. Architectural Impact: MLP vs CNN

We compare MLP and CNN feature extractors by utilizing the best-performing algorithm identified in previous trials for each visibility level: DQN for Full observability and PPO for Mask = 1 and Mask = 2. Both architectures were evaluated using the Default reward configuration and identical hyperparameters to isolate the impact of the architectural change.

All RL models significantly outperformed the heuristic baseline regardless of the feature extractor used. However, as shown in Fig. 13, the transition to CNNs provided limited benefits over MLPs. This suggests that for a small 7×7 grid, the spatial inductive bias of convolutional layers does not offer a substantial advantage over the dense connections of an MLP, as the state space is not complex enough to require hierarchical feature extraction.

Key observations from the evaluation process include:

- **Full and Mask = 2:** Both architectures achieve comparable median fruit counts. This indicates that at this

*Table 4.* Evaluation metrics for MLP-based policies in the 7×7 fully observable environment. Values reported as episodic means or percentage of total episodes.

| Model | Shaping | Real | Fruits | Steps | Wins % | Timeouts % | Wall % | Self % | Max |
|---|---|---|---|---|---|---|---|---|---|
| Heuristic | 11.09 | 5.38 | 11.10 | 46.42 | 0.0 | 0.0 | 31.0 | 69.0 | 18 |
| Random | -1.15 | -0.05 | 0.11 | 4.05 | 0.0 | 0.0 | 93.0 | 7.0 | 1 |
| DQN | 20.21 | 9.67 | 19.49 | 102.96 | 15.0 | 8.0 | 5.0 | 72.0 | 24 |
| A2C | 15.23 | 7.83 | 16.04 | 112.54 | 0.0 | 8.0 | 3.0 | 89.0 | 23 |
| PPO | 17.77 | 9.03 | 18.41 | 126.70 | 0.0 | 14.0 | 0.0 | 86.0 | 23 |

*Table 5.* Reward shaping configurations.

| Event | Default | NSP | Original | Aggressive | Survival |
|---|---|---|---|---|---|
| Fruit | 1.2 | 1.2 | 0.5 | 10.0 | 2.0 |
| Wall | -1.2 | -1.2 | -0.1 | -8.0 | -50.0 |
| Self | -1.2 | -1.2 | -0.2 | -8.0 | -50.0 |
| Step | -0.03 | 0.0 | -0.03 | -0.065 | 0.1 |
| Win | 3.0 | 3.0 | 1.0 | 35.0 | 1000.0 |
| Timeout | -0.5 | -0.5 | -0.5 | -4.0 | -1.0 |

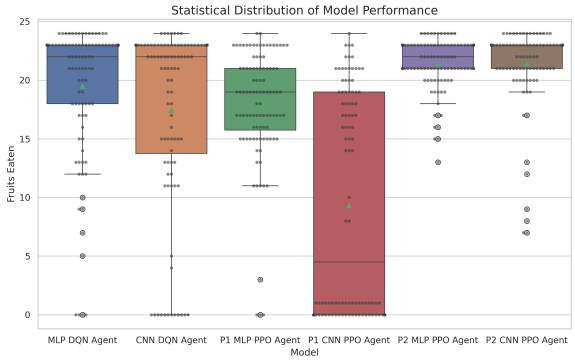


*Figure 13.* Performance comparison between MLP and CNN across observability settings. Both architectures consistently beat the heuristic baseline.

scale, the MLP is already capable of capturing the necessary spatial relationships to navigate effectively, making the added parameters of a CNN redundant.

- **Mask = 1:** CNN performance degrades significantly compared to the MLP. In such a restricted field of view, the convolutional filters lack sufficient spatial context to identify meaningful patterns, whereas the MLP's direct mapping remains more stable for reactive, pixel-level decision making.
- **DQN consistency:** In the fully observable case, both architectures retain high maximum performance and win rates. This confirms that the ability to complete the task is primarily driven by the algorithm's determinism and policy type rather than the specific method of feature extraction.

### 5.4. Robustness and Fine-Tuning (Obstacle Test)

To assess the generalization capabilities of the trained policies, we introduce a single stationary obstacle at a random location in each episode (Fig. 14). This environmental perturbation serves as a stress test for the agents' spatial awareness and adaptability.

*Figure 14.* Environment with a random obstacle.

The initial introduction of this obstacle causes a severe performance collapse (Fig. 15). Specifically, the DQN agent, which previously excelled in empty grids, reverts to nearly 100% wall collisions. This indicates a certain degree of overfitting to the base environment, where the agent learned fixed trajectories rather than a robust, reactive navigation logic.

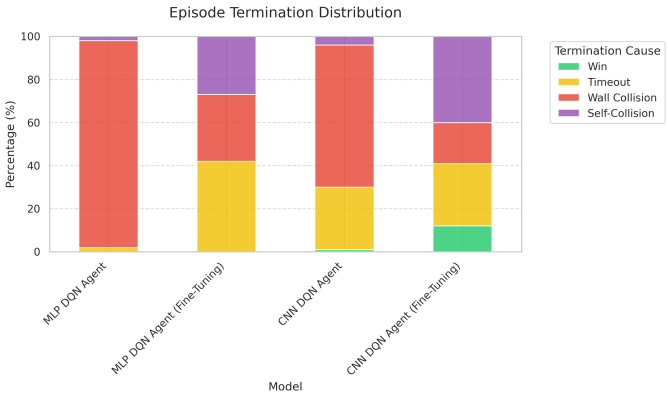


*Figure 15.* Episode outcomes with obstacle.

To mitigate this, we performed a fine-tuning phase (200k steps), leading to a partial recovery of performance. Key findings from this adaptation include:

- **Collision reduction:** The agents successfully learn to incorporate the obstacle into their obstacle-avoidance logic, significantly reducing the collision rate and outperforming the initial zero-shot results.
- **CNN Advantage in Adaptation:** Unlike previous trials where differences were marginal, the CNN architecture here proves superior, recovering a non-zero win rate (12%). This suggests that while MLPs are sufficient for static grids, CNNs provide a better spatial inductive bias when adapting to new, irregular constraints.
- **Efficiency Trade-off:** The recovery comes at a cost of efficiency; an increase in timeouts indicates that the agents have adopted a more conservative and hesitant policy to ensure survival in the presence of the obstacle, often at the expense of optimal fruit-seeking paths.

### 5.5. Multimedia Demonstrations and Extended Results

Due to the dynamic nature of grid-navigation tasks, visual demonstrations of the trained agents, including their behavior under partial observability and obstacle-perturbed environments, are provided as animated GIFs in the project repository. These visualizations offer a qualitative assessment of the pathfinding strategies learned by the DQN and PPO architectures. Furthermore, comprehensive performance data and specific metrics for each trial can be found in the `Training and Various Experiments` document. The complete collection of demonstrations, extended data, and source code can be accessed at:

https://github.com/AlejandroBernal95/RL_Snake_Final_Project

## 6. Technical Specifications

To ensure the reproducibility of our results, all experiments were conducted in a controlled hardware environment. The total execution time for the complete experimental suite, encompassing training for *630,000 steps* per model, sensitivity analyses, and evaluations, was *373.73 minutes* (22,424 seconds). As a benchmark for computational intensity, the primary DQN model for the 7×7 fully observable environment (Default configuration) required *19.38 minutes* (1,163 seconds) to converge. The hardware and software configurations utilized are detailed below:

- **Processor:** 13th Gen Intel® Core™ i7-13620H (16 logical cores).
- **Memory:** 32 GB RAM.
- **Graphics:** All training was CPU-bound.
- **Software Stack:** Python 3.13.12, utilizing the *Stable-Baselines3* library for RL implementations and *Gymnasium* for environment management.

## 7. Conclusion

This project provided a comprehensive evaluation of Reinforcement Learning architectures and reward structures within a 7×7 Snake environment. Through our experimental suite, we identified the DQN agent trained under the Default reward configuration (Section 5.1) as the superior model for the primary task. This agent demonstrated the highest terminal win rate and the most consistent mean reward per episode, effectively mastering the deterministic constraints of the fully observable grid.

Our analysis of architectural variations and information constraints revealed several key technical insights:

- **The Precision-Generalization Trade-off:** While the DQN agent excelled in the static, fully observable environment, it exhibited "learned helplessness" when faced with environmental perturbations (the obstacle test). In contrast, PPO and CNN-based architectures showed superior resilience and adaptability, suggesting they are better suited for dynamic or partially observable tasks despite lower terminal precision on the empty board.
- **Impact of Information Masking:** We observed that partial observability significantly hinders performance, particularly for CNNs with extremely limited vision radii (3×3). This highlights the importance of global state context for long-term path planning in constrained spaces.
- **Reward Shaping Criticality:** The catastrophic failure of the "Survival" toy model underscored the necessity of balanced reward signals. Without a step penalty to encourage efficiency, agents prioritize safe longevity over objective completion, leading to high timeout rates.

While we acknowledge that further hyperparameter tuning and more exhaustive reward optimization could yield even higher-performing "supreme" models, the current agents successfully met and exceeded our performance expectations. In conclusion, the DQN algorithm proved to be a robust and precise tool for grid-based navigation, provided that sufficient state information and temporal pressure are present during the learning phase.